

Bit, Byte ve Integer

BIL-304: Bilgisayar Mimarisi

Dersi veren öğretim üyesi:

Dr. Öğr. Üyesi Fatih Gökçe

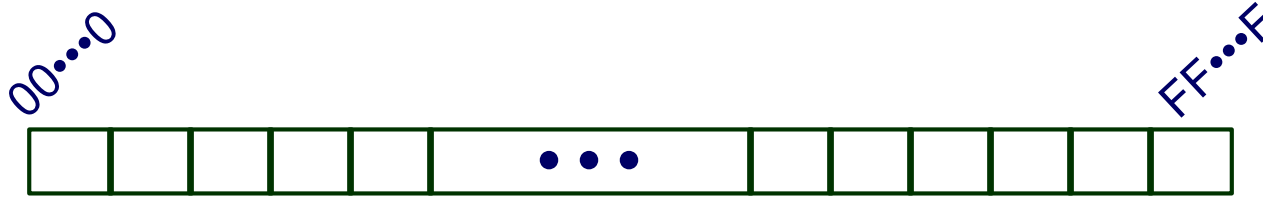
Ders kitabına ait sunum dosyalarından adapte edilmiştir: <http://csapp.cs.cmu.edu/>

Adapted from slides of the textbook: <http://csapp.cs.cmu.edu/>

Bit, Byte ve Integer

- Bilginin bitler ile ifade edilmesi
- Bit düzeyinde manipülasyonlar
- Integer
 - unsigned ve signed
 - Dönüştürme (conversion), tip dönüşümü (casting)
 - Genişletme (expanding), kırpma (truncating)
 - Toplama, işaret değiştirme (negation), çarpma, kaydırma
- **Bilginin hafızada saklanma şekilleri, pointer'lar, karakter dizileri (string'ler)**

Byte' a Yönelik Hafıza Organizasyonu



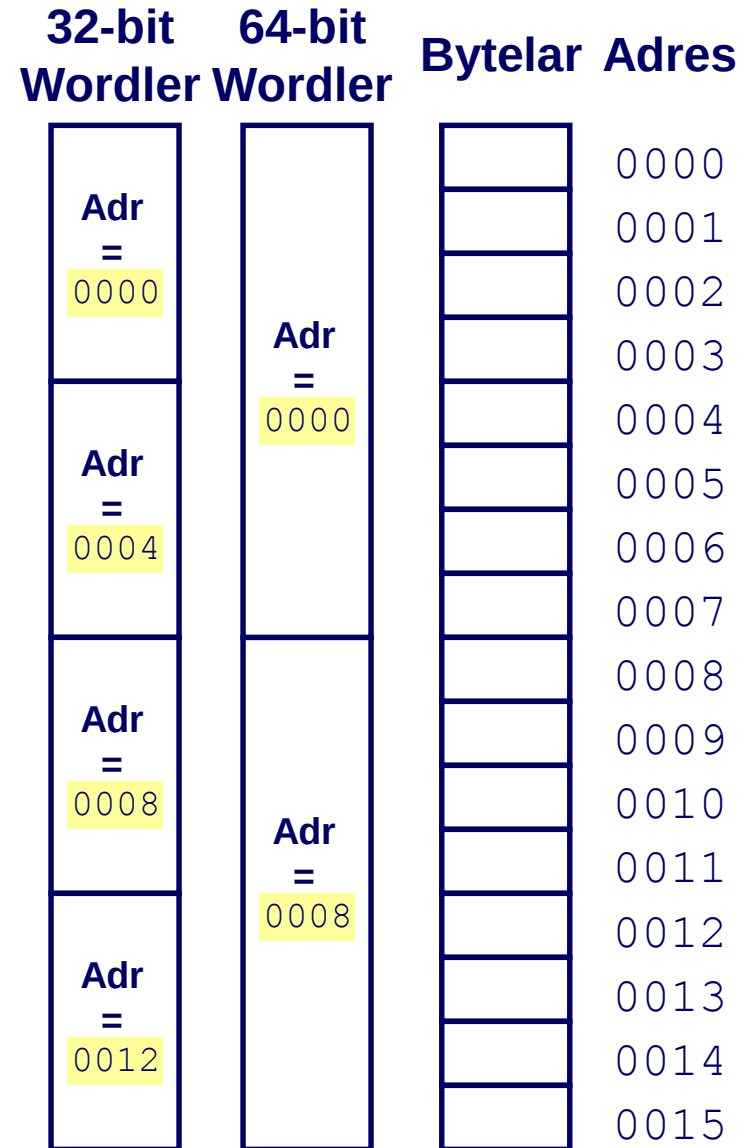
- **Programlar veriye adresleri aracılığıyla erişir**
 - Konsept olarak, hafızayı çok fazla miktarda byte'tan oluşan bir dizi olarak düşünebilirsiniz.
 - Gerçekte böyle olmasa da, bu şekilde kafanızda canlandırabilirsiniz.
 - Bir adres, bu diziye erişim sağlayan bir indeks gibidir.
 - ve, bir pointer (işaretçi) değişken bir adres değerini saklar.
- **Not: sistem her bir “process” e özel bir adres alana sağlar.**
 - Process'i çalıştırılmakta olan bir program olarak düşünebilirsiniz.
 - Bu nedenle, bir program kendi verisi üzerinde istediğini yapabilir, ama diğer programların verisine bir şey yapamaz.

Makine Word'leri (Kelimeleri, Sözcükleri)

- Her bilgisayar belli bir “Word Size” (Sözcük boyutu)’a sahiptir.
 - Genelde integer-değerli verinin nominal boyutudur.
 - ve adres değerlerinin boyutu.
- Yakın geçmişe kadar, birçok bilgisayar 32 bitlik (4 byte) word size’ a sahipti.
 - Bu da kullanılabilir adresleri 4GB (2^{32} byte)’a sınırlandırıyordu.
- Artık çoğu bilgisayar 64-bit word size’a sahip
 - Potansiyel olarak, 18 PB (petabyte)’lık bir hafızayı adresleyebilir.
 - Yani 18.4×10^{15}
- Bilgisayarlar yine de farklı tiplerde veri formatlarını destekliyor:
 - word size’ın küçük parçaları kadar veya katları kadar boyutlarda olabilirler.
 - boyutları daima tam sayıda bir byte değeridir.

Word'e yönelik Hafıza Organizasyonu

- Adresler Byte'ların bulunduğu yeri belirtir
 - Bir Word'deki ilk byte'ın adresini
 - Ardışık word'lerin adresleri 4 (32-bit) ve 8 (64-bit) byte farka sahiptir.



C'de veri tiplerinin tipik boyutları

C Veri Tipi	32-bit program	64-bit program
<code>char</code>	1	1
<code>short</code>	2	2
<code>int</code>	4	4
<code>long</code>	4	8
<code>int32_t</code>	4	4
<code>Int64_t</code>	8	8
<code>float</code>	4	4
<code>double</code>	8	8
<code>char *</code>	4	8

Byte Sıralaması

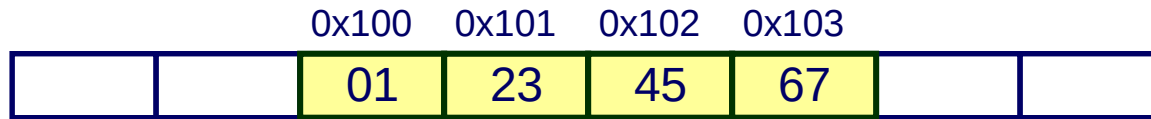
- **Peki, çok byte'lı bir word'ün byte'ları hafızada nasıl sıralanır?**
- **Kurallar:**
 - Big Endian (Büyük [Adres] Sonlu): Sun, PPC Mac, Internet
 - En anlamsız byte en yüksek adres değerine sahiptir.
 - Little Endian (Küçük [Adres] Sonlu) : x86, Android çalıştıran ARM işlemciler, iOS, and Windows
 - En anlamsız byte en düşük adres değerine sahiptir.

Byte Sıralaması Örneği

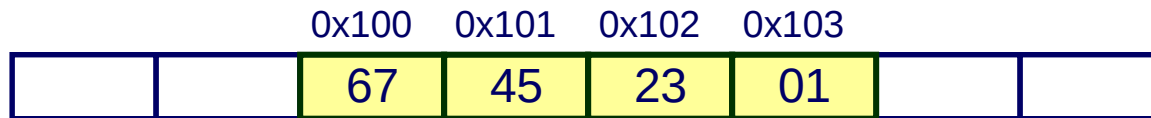
■ Örnek

- x değişkeni 4-bYTElık 0x01234567 değerine sahip
- &x ile gösterilebilen/erişilebilen x değişkeninin adresi 0x100

**Big Endian
(Büyük Sonlu)**



**Little Endian
(Küçük Sonlu)**



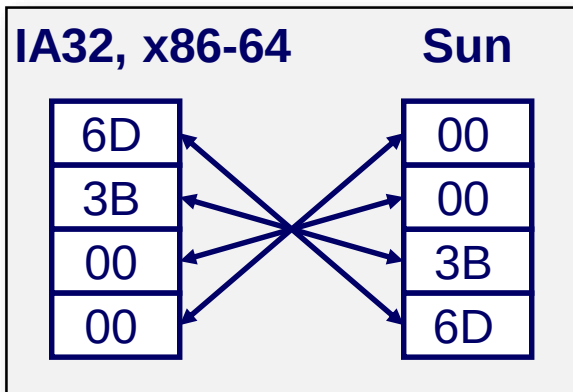
Hafızada Integer'lar

Onluk: 15213

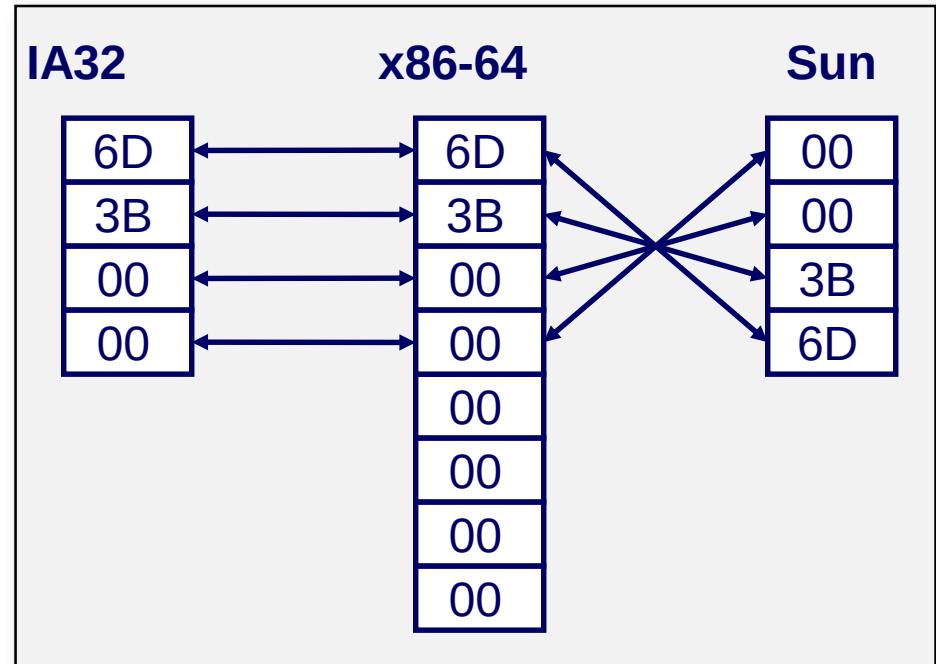
İkili: 0011 1011 0110 1101

Hex: 3 B 6 D

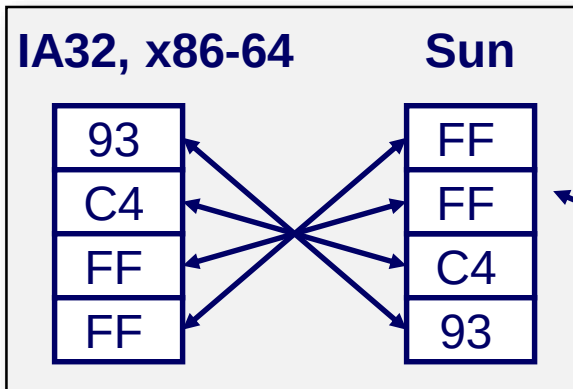
`int A = 15213;`



`long int C = 15213;`



`int B = -15213;`



2 Tümleyene göre gösterim

Bilginin hafızada saklanma şekillerinin incelenmesi

■ Veriyi oluşturan Byte'ları gösteren kod

- Pointer'ın unsigned char *'a dönüştürülmesi verinin byte dizisi olarak erişilmesine imkan sağlıyor:

```
typedef unsigned char *pointer;

void show_bytes(pointer start, size_t len){
    size_t i;
    for (i = 0; i < len; i++)
        printf("%p\t0x%.2x\n", start+i, start[i]);
    printf("\n");
}
```

Printf yönlendiricileri:

%p: pointer'i yazdırır

%x: Hexadecimal olarak yazdırır.

show_bytes'ın Çalışmasına Örnek

```
int a = 15213;  
printf("int a = 15213;\n");  
show_bytes((pointer) &a, sizeof(int));
```

Sonuç çıktıları (Linux x86-64):

```
int a = 15213;  
0x7fffb7f71dbc    6d  
0x7fffb7f71dbd    3b  
0x7fffb7f71dbe    00  
0x7fffb7f71dbf    00
```

Hafızada Pointer'lar

```
int B = -15213;  
int *P = &B;
```

Sun

EF
FF
FB
2C

IA32

AC
28
F5
FF

x86-64

3C
1B
FE
82
FD
7F
00
00

Farklı derleyiciler & bilgisayarlar nesnelere farklı adresleri atarlar. Hatta, aynı bilgisayarda farklı çalıştırmalarda bile bu adres atamaları değişebilir.

Hafızada String'ler

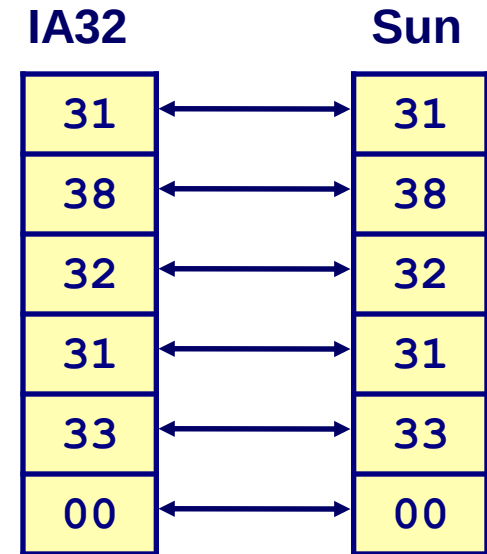
```
char S[6] = "18213";
```

■ C'de String'ler

- Karakter dizisi olarak ifade edilirler.
- Her karakter ASCII formatında kodlanmıştır.
 - Karakter setini standart 7-bit kodlanması
 - Karakter "0" ın kodu: 0x30
 - Genel olarak dijit i 'nin kodu: $0x30+i$
- Stringler null değer ile sonlandırılır.
 - Yani en son karakter = 0

■ Uyumluluk

- Byte sıralaması tüm bilgisayarlarda aynıdır.



Byte'ları Tersine çevrilmiş listelerin okunması

■ Disassembly – Makine dilinde verilen koddan assembly kodlarını çıkarma işi

- İkili makine kodunun text halinde ifadesi elde edilir.
- Makine kodunu okuyan bir program tarafından oluşturulabilir.

■ Örnek Parça

Adres	Komutların kodları	Assembly yorumlaması
8048365:	5b	pop %ebx
8048366:	81 c3 <u>ab 12 00 00</u>	add \$0x12ab,%ebx
804836c:	83 bb 28 00 00 00 00	cmpl \$0x0,0x28(%ebx)

■ Numaraların Çözülmesi

- Değer:
- 32 bite doldur:
- Byte'lara böl:
- Ters çevir:

0x12ab

0x000012ab

00 00 12 ab

ab 12 00 00

Kod Güvenlik Örneği

```
/* Kullanıcının erişebileceği verileri tutan Kernel hafıza bölgesi*/
#define KSIZE 1024
char kbuf[KSIZE];

/* En fazla maxlen kadar byte'ı kernel bölgesinden kullanıcının
istediği yere kopyala*/
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte sayısını gösteren len, buffer boyutu ve maxlen'den
    küçük olanın değerini alır*/
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

- Yukarıdaki kod yazılım hatası içeren FreeBSD'nin getpeername uygulamasına benzer bir koddur.
- Programlardaki açıkları bulmaya çalışan kişiler her zaman bulunur.

Tipik Kullanım

```
/* Kullanıcının erişebileceği verileri tutan Kernel hafıza bölgesi*/
#define KSIZE 1024
char kbuf[KSIZE];

/* En fazla maxlen kadar byte'ı kernel bölgesinden kullanıcının
istediği yere kopyala*/
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte sayısını gösteren len, buffer boyutu ve maxlen'den
    küçük olanın değerini alır*/
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

```
#define MSIZE 528

void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, MSIZE);
    printf("%s\n", mybuf);
}
```

Kötü Niyetli Kullanım

```
/* memcpy kütüphane fonksiyonunun tanımı */  
void *memcpy(void *dest, void *src, size_t n);
```

```
/* Kullanıcının erişebileceği verileri tutan Kernel hafıza bölgesi*/  
#define KSIZE 1024  
char kbuf[KSIZE];  
  
/* En fazla maxlen kadar byte'ı kernel bölgesinden kullanıcının  
istediği yere kopyala*/  
int copy_from_kernel(void *user_dest, int maxlen) {  
    /* Byte sayısını gösteren len, buffer boyutu ve maxlen'den  
    küçük olanın değerini alır*/  
    int len = KSIZE < maxlen ? KSIZE : maxlen;  
    memcpy(user_dest, kbuf, len);  
    return len;  
}
```

```
#define MSIZE 528  
  
void getstuff() {  
    char mybuf[MSIZE];  
    copy_from_kernel(mybuf, -MSIZE);  
    . . .  
}
```

Ne zaman Unsigned Kullanmalıyım?

- ***Modüler aritmetiği işletirken kullanabiliriz.***
 - 1 bit daha hassas aritmetik imkanı sağlar
- ***Bitleri küme halinde kullanacaksak kullanabiliriz.***
 - Lojik sağa kaydırma, işaret biti çoğaltılmaz.

C'de Integer Bulmacaları

foo ve bar
herhangi bir
değer dönen iki
fonksiyon

```
int x = foo();
int y = bar();
unsigned ux = x;
unsigned uy = y;
```

- $(x > 0) \ || \ (x-1 < 0)$
- $(x \ \& \ 7) \ != \ 7 \ || \ (x \ll 29 < 0)$
- $(x * x) \geq 0$
- $x < 0 \ || \ -x \leq 0$
- $x > 0 \ || \ -x \geq 0$
- $x + y == uy + ux$
- $x \sim y + uy * ux == -x$

Verilen ifadeler her zaman
DOĞRU mu döner, **YANLIŞ**
dönebilecekleri durum varsa bu
hangi durumdur?